

SpeakScore System Architecture

Project: AI Pronunciation Checker for Livo AI SWE Assessment

Author: Rahul Das

Date: July 12, 2026

Version: 3.0

Repository: [pronunciation-checker-web-app](#)

1. Executive Summary

SpeakScore is a full-stack pronunciation assessment application for English learners. A user can either record speech in the browser or upload an audio file, then receive a structured analysis with overall score, accuracy score, fluency score, word-level feedback, phonetic "you said vs correct" comparisons, and short improvement tips.

The architecture intentionally separates three concerns:

- Interactive client experience:** Next.js + React handles recording, upload validation, consent, and visualization of word-level feedback.
- Audio and AI orchestration:** FastAPI validates temporary audio, calls Groq Whisper for speech-to-text, enriches the transcript with phoneme data, asks Groq Llama for pronunciation reasoning, and computes deterministic final scores.
- Privacy-first processing:** Audio is processed only for the active request, stored only as a temporary server file, and deleted in the API cleanup path.

The strongest engineering choice is the **multi-signal scoring pipeline**. Instead of relying only on speech-to-text output, SpeakScore combines acoustic confidence, text/reference alignment, phoneme lookup, and LLM-based pronunciation reasoning. This produces more useful feedback while keeping the system lightweight enough for a hosted assessment project.

2. System Context

```
User Browser
- Uploads or records audio
- Accepts DPDP consent
- Selects read-aloud or free-speech mode
- Views scores and word-level feedback

    HTTPS multipart/form-data
    audio_file + optional expected_text
    |
    v

FastAPI Backend on Render
- Validates size and duration
- Temporarily stores audio for decoding
- Sends audio to Groq Whisper
- Extracts phonemes with CMU Dictionary + g2p-en
- Sends transcript and phoneme context to Groq Llama
- Computes final response
- Deletes temp audio file

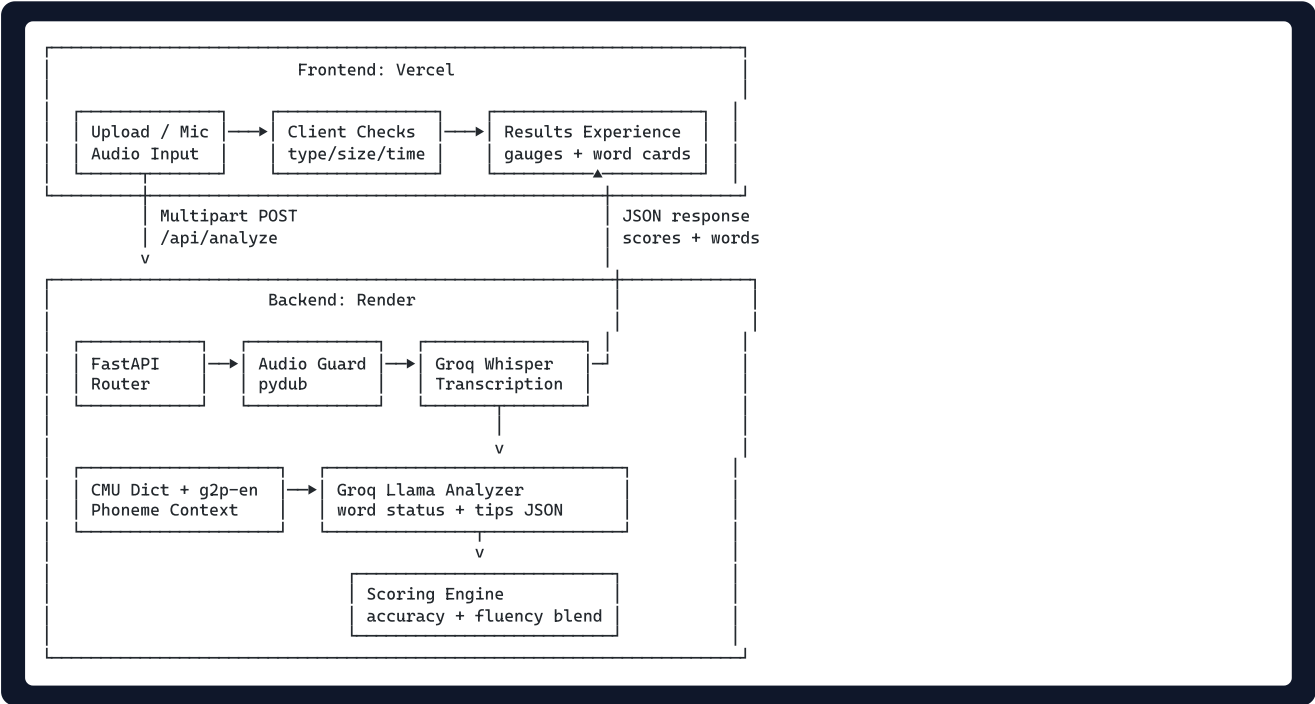
    |
    v

Groq AI Services
- whisper-large-v3: transcription, word timestamps, segment confidence
- llama-3.3-70b-versatile: JSON pronunciation analysis
```

Key Runtime Boundaries

Boundary	Responsibility	Main Files
Browser UI	Recording, upload, validation, consent, results rendering	<code>frontend/src/app/page.tsx</code>
API client	Multipart request and typed response contract	<code>frontend/src/lib/api.ts</code>
API router	Request lifecycle and cleanup	<code>backend/routers/analyze.py</code>
Audio validation	File size, duration, temporary file handling	<code>backend/utils/audio.py</code>
Transcription	Groq Whisper integration	<code>backend/services/transcription.py</code>
Phoneme enrichment	CMU pronunciation lookup and g2p fallback	<code>backend/services/pronunciation.py</code>
LLM analysis	Prompting and structured JSON parsing	<code>backend/services/llm_analyzer.py</code>
Scoring	Deterministic accuracy, fluency, and overall score	<code>backend/services/scoring.py</code>

3. High-Level Component Architecture



4. Technology Stack

Layer	Technology	Why It Fits
Frontend	Next.js 16.2.10, React 19.2.4, TypeScript	Modern app-router UI, typed API contracts, easy Vercel deployment
Styling and UX	Tailwind CSS 4, lucide-react	Fast responsive UI, clear iconography, polished dark theme
Browser audio	MediaRecorder API, file input, HTML audio metadata	Supports both microphone recording and uploaded files
Backend	FastAPI, Python 3.11 on Render	Simple async API surface, strong Python audio/ML ecosystem
Audio parsing	pydub + FFmpeg	Validates duration and catches malformed audio before AI calls
Speech-to-text	Groq <code>whisper-large-v3</code>	Fast transcription with verbose JSON, word timestamps, and segment log probabilities
Pronunciation context	<code>pronouncing</code> , CMU Dict, <code>g2p-en</code> , NLTK	Lightweight phoneme generation with fallback for unknown words
LLM reasoning	Groq <code>llama-3.3-70b-versatile</code>	Fast structured JSON analysis and human-readable feedback
Deployment	Vercel frontend, Render backend	Low-friction split deployment for a full-stack assessment

5. User Modes

Read-Aloud Mode

The user selects one of the built-in reference passages and reads it aloud. The backend receives both the audio and the expected text, allowing the LLM analyzer to classify words as:

- `correct`
- `substitution`
- `insertion`
- `deletion`
- `unclear`

This mode is best for measuring pronunciation accuracy against a known target sentence.

Free-Speech Mode

The user speaks naturally without a reference passage. The backend sends an empty `expected_text`, and the LLM prompt switches to clarity-focused evaluation. In this mode, the system avoids reference-only labels such as deletion and focuses on articulation, confidence, pacing, and unclear words.

6. Request Lifecycle

1. User records or uploads audio in the browser.
2. Frontend performs early validation:
 - Max file size: 10 MB
 - Accepted types: wav, mp3, m4a, ogg, webm, mp4
 - Browser duration check where metadata is available
 - Recording timer caps microphone recordings at 45 seconds
3. User gives explicit data-processing consent.
4. Frontend submits:
POST {NEXT_PUBLIC_API_URL}/api/analyze
Content-Type: multipart/form-data
Fields:
 - audio_file: binary audio
 - expected_text: reference text for read-aloud mode, omitted/empty for free speech
5. Backend reads the upload into memory and writes a temporary file.
6. Audio guard validates:
 - Max server-side file size: 10 MB
 - Minimum duration: 1 second
 - Maximum duration: 45 seconds
 - Audio decodability via pydub/FFmpeg
7. Groq Whisper transcribes the temporary file with:
 - model: whisper-large-v3
 - response_format: verbose_json
 - timestamp_granularities: word and segment
 - provider timeout: 45 seconds
8. Backend extracts phonemes for expected and transcribed words:
 - CMU Dictionary first
 - g2p-en fallback for out-of-vocabulary words
9. Groq Llama receives the transcript, word timestamps, avg_logprob, no_speech_prob, expected phonemes, and transcribed phonemes.
 - provider timeout: 30 seconds
10. Llama returns strict JSON:
 - overall feedback
 - focus areas
 - word-by-word status
 - phonetic respellings
 - actionable pronunciation tips
11. Scoring service computes final scores.
12. API returns `AnalyzeResponse` JSON.
13. `finally` cleanup deletes the temporary audio file.

7. API Contract

Endpoint

Method	Path	Purpose
GET	/	Health check
POST	/api/analyze	Analyze one audio recording

Request

```
POST /api/analyze
Content-Type: multipart/form-data

audio_file=<binary audio>
expected_text=<optional reference passage>
```

Response

```
{
  "overall_score": 82,
  "fluency_score": 85,
  "accuracy_score": 80,
  "overall_feedback": "Your speech was clear overall...",
  "focus_areas": ["word endings", "th sounds"],
  "transcribed_text": "The sun is shining brightly today",
  "words": [
    {
      "word": "through",
      "status": "substitution",
      "expected_phonemes": null,
      "issue": "Extra final sound added after the vowel.",
      "tip": "End the word cleanly after the oo sound.",
      "you_said": "throo-w",
      "correct_pronunciation": "throo",
      "start_time": 1.24,
      "end_time": 1.62,
      "confidence": null
    }
  ]
}
```

Error Handling

Failure	Response Behavior
File too large	413 with max-size message
Too short or too long	400 with duration message
Invalid/undecodable audio	400 with pydub validation detail
Whisper/Groq transcription failure	502 with a user-safe retry message
Llama/Groq analysis failure	Fallback result from transcript alignment and acoustic score
Frontend network/API failure	Error banner shown in the upload flow

8. Pronunciation Analysis Pipeline

SpeakScore uses a multi-signal design because pronunciation quality is not captured by a single metric.

Signal 1: Acoustic Confidence

Whisper returns segment-level `avg_logprob`. The scoring service averages log probabilities across all segments and maps them to a fluency score:

Avg Logprob	Fluency Score	Interpretation
<code>>= -0.1</code>	95	Excellent clarity
<code>>= -0.2</code>	85	Very good
<code>>= -0.3</code>	75	Good
<code>>= -0.5</code>	60	Below average
<code>>= -0.7</code>	40	Poor
<code>< -0.7</code>	0-30	Very unclear or mumbled

Signal 2: Reference Alignment

In read-aloud mode, the expected passage and Whisper transcript are both provided to the analyzer. This lets the system identify skipped words, extra words, and substitutions.

Signal 3: Phoneme Context

The backend converts words to ARPAbet phonemes before the LLM call. This gives the model concrete sound-level context instead of asking it to infer pronunciation only from spelling.

Signal 4: LLM Expert Review

The prompt instructs the LLM to:

- accept major English varieties as valid
- focus on intelligibility rather than accent conformity
- be generous when Whisper confidently transcribed the word
- return strict JSON only
- provide simple phonetic respelling, not IPA, for learner readability

Final Score Formula

```
accuracy_score = correct_words / total_words * 100
fluency_score  = normalized Whisper avg_logprob
overall_score  = (accuracy_score * 0.60) + (fluency_score * 0.40)
```

The scoring step is deterministic after the LLM JSON is received. This keeps the final numeric blend explainable and easy to tune.

9. Frontend Architecture

```
frontend/src/
├── app/
│   ├── layout.tsx      Root layout, metadata, nav, footer
│   ├── page.tsx        Main product flow and all UI states
│   ├── globals.css     Theme, animations, card styles
│   └── privacy/page.tsx DPDP-oriented privacy and consent withdrawal
├── lib/
│   ├── api.ts          Fetch client and response interfaces
│   ├── audio-utils.ts  Type, size, duration helpers
│   └── constants.ts     Built-in read-aloud passages
```

UI State Model

State	Behavior
Idle	User chooses read-aloud/free-speech mode, uploads, drags, or records
Recording	Microphone stream captured with timer and max-duration guard
File Ready	Audio preview, file metadata, analyze action
Consent Required	Modal blocks analysis until the user agrees
Analyzing	Loading state while backend calls AI providers
Results	Score gauge, feedback summary, word cloud, detailed issue cards
Error	User-readable validation or API failure message

UX Details That Help the Assessment

- Read-aloud passages are grouped by difficulty, showing product thought beyond a raw upload tool.
- The result view makes errors scannable through color, legend, word cards, and specific tips.
- The privacy page includes withdrawal of local consent by clearing `localStorage`.
- The UI supports both browser-recorded WebM audio and uploaded common audio formats.

10. Backend Architecture

```
backend/
├── main.py           FastAPI app, CORS, NLTK resource setup
├── config.py         Environment-backed settings
├── routers/
│   └── analyze.py    Request orchestration and cleanup
├── models/
│   └── schemas.py    Pydantic response models
├── services/
│   ├── transcription.py Groq Whisper call
│   ├── pronunciation.py CMU/g2p phoneme extraction
│   ├── llm_analyzer.py  Prompt, Groq Llama call, JSON parsing
│   └── scoring.py       Deterministic scoring logic
├── utils/
│   └── audio.py       pydub validation and temp file creation
```

Backend Design Notes

- **Single orchestration route:** `/api/analyze` is intentionally simple and readable for assessment review.
- **Temporary-file bridge:** pydub and the Groq file upload both work naturally with file paths, so the system writes a temporary file and deletes it after processing.
- **Pydantic output schema:** The API response shape is explicit and aligned with frontend TypeScript interfaces.
- **Provider isolation:** Whisper, phoneme extraction, LLM analysis, and scoring live in separate service modules, making future replacement easier.
- **Concurrency-aware orchestration:** The async route offloads blocking audio/provider work to FastAPI's threadpool, allowing other requests to continue being accepted.
- **Graceful degradation:** If the LLM analyzer is unavailable after transcription succeeds, the backend falls back to deterministic transcript/reference alignment.
- **Provider timeouts:** Whisper and Llama requests use explicit timeouts so failed upstream calls do not hold server workers indefinitely.

11. Data Privacy and DPDP-Oriented Controls

SpeakScore processes voice recordings, so the architecture treats privacy as a product requirement rather than an afterthought.

Implemented Controls

Principle	Implementation
Consent	The frontend blocks analysis behind an explicit consent modal.
Purpose limitation	Audio is used only for pronunciation analysis.
Data minimization	The response contains scores, transcript text, and feedback; the app does not persist audio.
Storage limitation	Backend writes audio only to a temporary file for validation and provider upload.
Cleanup	<code>finally</code> block attempts to delete the temporary audio file after success or failure.
Withdrawal	Privacy page lets the user clear local consent state from the browser.
Processor disclosure	Privacy page states that Groq is used for transcription and LLM analysis.

Data Lifecycle

```
Browser audio
-> HTTPS upload
-> temporary backend file
-> Groq processing request
-> JSON analysis response
-> temporary file deletion
-> frontend-only display of result
```

What Is Not Stored

- No user accounts
- No database records
- No server-side result history
- No long-term audio files
- No app-owned object storage bucket

This is a strong fit for the assessment because it demonstrates deliberate minimization: the system does not add auth, profiles, history, or storage until there is a clear product reason and a stronger compliance design.

12. Deployment Architecture

```

GitHub Repository
|
+--> Vercel
|   - Builds frontend from `frontend/`
|   - Uses `NEXT_PUBLIC_API_URL`
|   - Serves Next.js app over HTTPS
|
+--> Render
|   - Builds Python backend
|   - Installs `backend/requirements.txt`
|   - Starts `uvicorn main:app --host 0.0.0.0 --port $PORT`
|   - Receives `GROQ_API_KEY` as a managed secret

```

Live Services

Service	URL
Frontend	https://pronunciation-checker-web-app.vercel.app/
Backend	https://livo-pronunciation-backend.onrender.com/

Environment Variables

Name	Location	Purpose
<code>NEXT_PUBLIC_API_URL</code>	Vercel/frontend	Points the browser to the deployed FastAPI backend
<code>GROQ_API_KEY</code>	Render/backend	Authenticates Whisper and Llama provider calls

13. Reliability, Scalability, and Error Handling

Current Strengths

- Client-side validation reduces invalid backend calls.
- Backend validation is still authoritative, preventing oversized or invalid files from reaching AI services.
- The app caps recordings and backend audio duration to keep latency and cost bounded.
- The AI pipeline uses Groq for low-latency inference.
- The backend separates provider calls from scoring so performance bottlenecks are easy to isolate.
- Blocking audio parsing and Groq SDK calls are offloaded through FastAPI's threadpool so the async event loop can continue accepting other requests.
- Groq calls have explicit timeouts: 45 seconds for transcription and 30 seconds for LLM analysis.

Scalability and Concurrency

The backend does not have a hard-coded user limit. In practice, concurrency is bounded by the deployed Render instance size, Uvicorn worker/threadpool configuration, memory, request duration, and Groq provider rate limits.

For this assessment MVP, the system is designed for **multiple concurrent audio analyses** rather than one blocking request at a time:

- FastAPI accepts requests asynchronously.
- Upload reading uses the async `UploadFile` flow.
- Synchronous work such as pydub duration checks, Groq Whisper calls, phoneme extraction, Llama calls, and scoring is moved to the server threadpool.
- File size and duration caps, 10 MB and 45 seconds, prevent a single user from monopolizing CPU, memory, or provider latency for too long.
- The API is stateless, so horizontal scaling is straightforward: additional Render instances or workers can process independent requests without shared session state.

A production deployment would tune `uvicorn` workers, threadpool capacity, request timeouts, and Groq rate limits after load testing. The expected bottleneck is the external AI pipeline latency, not local Python routing.

Error Handling and Fallbacks

The backend uses layered failure handling:

Failure Point	Behavior
Unsupported or corrupt audio	Rejects before AI calls with a <code>400</code> validation error
Oversized audio	Rejects with <code>413</code> before provider calls
Whisper/Groq transcription unavailable	Returns <code>502</code> with a clear retry message because no transcript exists
Llama/Groq analysis unavailable	Falls back to deterministic transcript/reference alignment and still returns scores
Provider latency spike	Request-level Groq timeouts prevent indefinite worker blocking
Temporary file cleanup	Runs in <code>finally</code> , even when validation, transcription, or analysis fails
Frontend API failure	Displays a readable error banner and keeps the user in the upload flow

This means the user is not left with a raw stack trace. The only unrecoverable provider failure is transcription, because without speech-to-text output the system has no reliable text to score.

Known MVP Limits

Area	Current State	Production Improvement
CORS	Backend currently allows all origins for demo convenience	Restrict to Vercel production and preview domains
Rate limiting	Not implemented	Add IP/user-level rate limits to protect provider quota
Observability	Basic exception surfacing only	Add structured logs, request IDs, timing metrics
Provider resilience	Whisper failures return <code>502</code> ; Llama failures use deterministic fallback; provider calls use explicit timeouts	Add retries, circuit breakers, and secondary providers
Test coverage	Not visible in repository	Add unit tests for scoring/audio guards and integration tests for API contract
Audio precision	Uses ASR confidence and LLM reasoning	Add forced alignment or GOP scoring for phoneme-level acoustic evidence

14. Model Choices and Trade-Offs

Decision	Chosen	Trade-Off
Speech-to-text	Groq Whisper	Fast and high quality, but still ASR-based rather than a dedicated pronunciation engine
Pronunciation reasoning	Groq Llama JSON output	Produces human-friendly feedback, but output quality depends on prompt discipline
Phoneme source	CMU Dictionary + g2p-en	Lightweight and deployable, but not as precise as forced alignment
Backend language	Python/FastAPI	Excellent for audio and ML libraries, separate runtime from the JS frontend
Persistence	No database	Strong privacy posture, but no long-term progress tracking
Deployment	Vercel + Render	Fast assessment-friendly deployment, but limited free-tier scaling controls

15. Future Roadmap

1. **Forced alignment and GOP scoring:** Add Montreal Forced Aligner, Wav2Vec2, or a specialized pronunciation model to produce phoneme-level acoustic scores.
 2. **Streaming analysis:** Replace full-file multipart uploads with chunked upload or WebSocket streaming for faster feedback.
 3. **Authenticated progress tracking:** Add optional accounts and consented result history for learners who want progress analytics.
 4. **Teacher dashboard:** Let instructors assign passages and review aggregate class pronunciation trends.
 5. **Production security hardening:** Restrict CORS, add rate limits, add request tracing, and use provider timeouts/retries.
 6. **Multi-language support:** Extend the architecture to additional languages using multilingual transcription and language-specific phoneme resources.
-

16. Why This Architecture Is Strong for Shortlisting

- It solves the actual product problem end to end: audio input, AI analysis, scoring, feedback, privacy, and deployment.
 - It uses a modular backend where each service has a clear responsibility.
 - It avoids unnecessary storage and account complexity in an assessment context.
 - It explains how the score is computed instead of hiding everything behind an LLM.
 - It acknowledges real MVP trade-offs and gives a credible path to production.
 - It includes privacy and consent as part of the system design, which is especially relevant for voice data.
-

Appendix: Implementation Evidence

Claim	Evidence in Code
Browser upload and recording	<code>frontend/src/app/page.tsx</code>
Client audio checks	<code>frontend/src/lib/audio-utils.ts</code>
API request contract	<code>frontend/src/lib/api.ts</code>
FastAPI route	<code>backend/routers/analyze.py</code>
Temporary-file validation	<code>backend/utils/audio.py</code>
Whisper transcription	<code>backend/services/transcription.py</code>
Llama pronunciation prompt	<code>backend/services/llm_analyzer.py</code>
Deterministic scoring formula	<code>backend/services/scoring.py</code>
Privacy page and consent withdrawal	<code>frontend/src/app/privacy/page.tsx</code>
Render deployment config	<code>render.yaml</code>
Vercel deployment config	<code>frontend/vercel.json</code>